API Integration Guide

Payouts, beneficiaries, balances and OTC orders: authentication, request signing, callbacks, error handling and the full endpoint reference.

Version 2026-10-23

Contents

OVERVIEW

1. Introduction

GETTING STARTED

1. HTTP status codes
2. Authentication
3. Error codes
4. Best practices
5. Callbacks integration

API REFERENCE

1. Payouts
2. Beneficiaries
3. Balances
4. OTC quotes & orders
5. Utilities

Introduction

Integrate the payment gateway — payouts, beneficiaries, balances, and OTC orders.

The payment gateway lets you securely integrate payment processing into your application using an **Access Key** and **Secret Key**. This enables seamless online payments, beneficiary management, fiat balances and OTC FX directly from your platform.

To access these APIs you must have an **active access key and secret key**, issued through the **merchant dashboard**.

Quickstart

Authentication
HMAC SHA256 request signing with your access key, nonce, and secret.

HTTP status codes
Standard codes returned across every endpoint, with their meaning.

Payouts
Initiate bank or UPI payouts and track their lifecycle.

OTC quotes & orders
Request FX prices and execute conversion orders.

Environments

The API has two independent environments. They share the same API surface — point your client at sandbox while building, switch to production when you go live.

The base URL for each environment (`{BASE_URL}` in these docs) is shared with you alongside your API credentials.

Environment	Base URL
Sandbox	Provided with your credentials
Production	Provided with your credentials

HTTP status codes

Standard HTTP codes returned across every endpoint.

The APIs use standard HTTP status codes to indicate the outcome of each request.

Code	Name	Meaning
200	Success	Request was processed successfully.
400	Bad Request	Invalid request syntax or missing required parameters.
401	Unauthorized	Authentication failed or credentials were not provided.
403	Forbidden	Access denied due to insufficient permissions.
404	Not Found	Requested resource or endpoint does not exist.
405	Method Not Allowed	The HTTP method used is not allowed for the specified endpoint.
422	Unprocessable Entity	Validation error or business rule failure (for example, insufficient balance).
500	Internal Server Error	An unexpected error occurred on the server.
503	Service Unavailable	Rate limit exceeded. Returned with an empty body.

Rate limit

The API currently allows **20 requests per second**. Requests above the limit receive `503` with an empty body — wait briefly and retry.

Authentication

Sign every request with your access key, a nonce, and an HMAC SHA256 signature.

Authentication is required for all **private APIs** and is performed using API credentials — an **access key** and **secret key**. Public APIs do not require authentication.

How to retrieve API keys

1 Sign in to your account

Sign in to the **merchant dashboard**.

2 Enable Two-Factor Authentication

Navigate to **Dashboard** → **Settings** → **Security** and turn on 2FA.

> 2FA must be enabled before generating or managing API keys.

3 Generate keys

Go to **Settings** → **API** to view or generate a new key pair. The secret key is shown once — store it in a secret manager (AWS Secrets Manager, GCP Secret Manager, Doppler, Vault).

Required headers (private APIs)

Every authenticated request must include the following headers:

```
X-Auth-Apikey
X-Auth-Nonce
X-Auth-Signature
Content-Type: application/json
```

Generating the signature

For every request, generate a fresh **nonce** (millisecond timestamp) and an HMAC SHA256 **signature** of `nonce + accessKey`, keyed with your secret. A nonce more than **50 seconds** old is rejected with `authz.nonce_expired`, so generate it immediately before sending and keep your server clock in sync.

> Code samples below use `BASE_URL` as a placeholder — replace it with the base URL for your environment, which is shared with you alongside your API credentials. See Environments.

sign.js

```
import CryptoJS from "crypto-js";

const BASE_URL = "your-base-url-here";
const nonce = Date.now().toString();

const accessKey = "your-access-key-here";
const secretKey = "your-secret-key-here";

const sigString = nonce + accessKey;
const signature = CryptoJS
  .HmacSHA256(sigString, secretKey)
  .toString();

fetch(`${BASE_URL}/account/balances/inr`, {
  method: "GET",
  headers: {
    "X-Auth-Apikey": accessKey,
    "X-Auth-Nonce": nonce,
    "X-Auth-Signature": signature,
    "Content-Type": "application/json"
  }
})
  .then(res => res.json())
  .then(data => console.log(data))
  .catch(err => console.error(err));
```

Security notice. Your API keys grant significant access. Never expose your secret key in client-side code, public repositories, or shared environments. Rotate keys periodically and on any personnel change.

Error codes

Machine-readable error codes returned by authentication and routing.

When an error occurs, the API returns a structured response with a machine-readable code plus a human-readable message.

Authentication errors

Authentication errors are returned with HTTP `401` before the request reaches the API, so **nothing is created** — fix the cause and retry.

Code	Meaning
`authz.invalid_api_key_headers`	One of the `X-Auth-*` headers is missing.
`authz.nonce_not_valid_timestamp`	The nonce is missing, zero, or not a number.
`authz.nonce_expired`	The nonce is more than 50 seconds old.
`authz.apikey_not_found`	The API key does not exist, was deleted, or is invalid.
`authz.apikey_not_active`	The API key is inactive or disabled.
`authz.apikey_scope_not_permitted`	The API key is not allowed to call this endpoint.
`authz.apikey_ip_not_permitted`	The request IP is not on the API key's allow-list.
`authz.invalid_signature`	The provided request signature is invalid.
`authz.invalid_session`	The account that owns the API key is not active.
`authz.invalid_permission`	The account does not have permission for this action.
`authz.permission_denied`	Access to this endpoint is blocked.

Routing errors

Code	Meaning
`404`	The endpoint path does not exist or the URL is incorrect.

See HTTP status codes for the full list of HTTP codes returned by the API.

Best practices

Guidelines for building reliable integrations on top of the API.

Do not rely on callbacks alone

Callback responses should only be treated as **triggers**. They are not the source of truth. On every callback event, call the Get Payout Status API to fetch the actual and final payout status.

Always use a unique order ID

Each payout request must include a unique `client_reference_id`. This acts as an **idempotency key** to prevent duplicate payouts when retrying.

Implement periodic status checks

Run a cron job (every 5 minutes is a good starting point) to sweep payout statuses. This ensures reliability if a callback event is missed or delayed.

Do not assume a payout has failed

Temporary errors do not always mean failure. Always query payout status using the same `client_reference_id` before retrying.

Treat callbacks as triggers, not as truth

Callbacks may be delayed or out of date. Always reconcile with the status API before acting on a callback payload.

Do not assume "no response" means failure

Network or server issues may cause temporary outages. A payout should be considered failed **only** when the Get Payout Status API returns a successful response with `payout_status: "rejected"`.

Callbacks integration

Receive signed event notifications when payouts and beneficiaries change state.

The platform sends real-time notifications to your application when important events occur — payout state changes, beneficiary verification completes, and so on. Each notification is an **HTTP POST** request to your configured endpoint with a JSON payload.

How to configure webhooks

1. ### Sign in to your account

   Sign in to the **merchant dashboard**.

2. ### Open the webhooks panel

   Navigate to **Dashboard** → **Settings** → **Security** → **Webhooks**.

3. ### Register endpoints

   Enter the public HTTPS URL for each webhook type you want to receive.

4. ### Copy the signing secret

   Copy the **shared signing secret** from the same Webhooks section and store it in your secret manager. You will need it to verify incoming requests.

What you'll receive

- **Method:** `POST`
- **Content-Type:** `application/json`
- **Header:** `X-Signature`
- **Payload:** JSON

You can register a **separate URL per event type** (payout, bank-beneficiary, UPI-beneficiary), so different event families can be delivered to different endpoints.

Delivery, acknowledgement & retries

- **Acknowledgement:** respond with any **HTTP 2xx**. A response body is **not** required — the status code alone acknowledges receipt.
- **Timeouts:** ~**10 s** read/response, ~**1 s** to connect.
- **Retries:** up to **2** retries (3 attempts total), starting at ~**0.5 s** with exponential backoff (≈0.5 s, then ≈1 s).
- **After the final attempt:** delivery is **best-effort** — if all attempts fail, the event is **not** re-queued and there is no dead-letter replay.

Because delivery is best-effort, treat callbacks as a **fast notification, not a guaranteed ledger**: keep your handler **idempotent** (dedupe on `payout_id` / `client_reference_id`), return `2xx` quickly and process asynchronously, and **reconcile via the status APIs** as the source of truth.

Payout status update

Payout callbacks are delivered on the **terminal** events `succeed`, `refunded`, and `rejected` (the in-flight `processing` state is **not** delivered as a webhook). The body carries these **10 fields**:

```
{
  "payout_id": "TID251224095503GMA37C3EE",
  "bank_currency": "inr",
  "beneficiary_id": "WBID251224092108GMC9757E",
  "client_reference_id": "BIGBULL0001-2",
  "amount": "100.0",
  "fee": "0.0",
  "payout_status": "succeed",
  "bank_transaction_id": "UTR2412240955030001",
  "status_code": "COMPLETED",
  "status_message": "Payout Completed"
}
```

On a **rejected** payout the same 10 fields are sent with: `payout_status: "rejected"`, `bank_transaction_id: null`, `status_code: "EXT: <reason> | Auto rejected"`, `status_message: null`.

> The payout callback does **not** include `sender_name`, `created_at`, or `completed_at` (those appear on the `GET /account/payouts/status` response, not the webhook), nor any sender-identity or OTC fields. Correlate to that data via `client_reference_id` / `payout_id`.

Bank-account beneficiary status update

Core body is **8 fields**; the four `verification_data_*` fields are added **only when a penny-drop verification ran**. Example (verified):

```
{
  "beneficiary_id": "WBID251024044324GM560AAE",
  "bank_currency": "inr",
  "bank_ifsc_code": "SBIN0000112",
  "account_number": "1234567890",
  "account_holder_name": "AMIT DOE",
  "status_code": "VERIFIED",
  "status_message": "Verification Completed",
  "verification_status": "active",
  "verification_data_bank_ref_no": "BRN-20251024T044324777-93VQR9",
  "verification_data_bank_beneficiary_name": "USER_NAME",
  "verification_data_bank_transaction_remark": "Transaction Successful",
  "verification_data_bank_transaction_status": "VERIFIED"
}
```


On a **rejected** verification, `verification_status: "rejected"`, `status_code` carries the reason (e.g. `ACCOUNT_BLOCKED`, `INVALID_ACCOUNT_NUMBER`), and the `verification_data_*` fields are `null` except `verification_data_bank_transaction_status: "FAILED"`.

UPI beneficiary status update

Core body is **6 fields** (note: **no** `account_holder_name`); the four `verification_data_*` fields are added **only when a penny-drop verification ran**. Example (verified):


```
{
  "beneficiary_id": "WBID251024044324GM560AAE",
  "bank_currency": "inr",
  "vpa": "user@upi",
  "status_code": "VERIFIED",
  "status_message": "Verification Completed",
  "verification_status": "active",
  "verification_data_bank_ref_no": "BRN-20251024T044324777-93VQR9",
  "verification_data_bank_beneficiary_name": "USER_NAME",
  "verification_data_bank_transaction_remark": "Transaction Successful",
  "verification_data_bank_transaction_status": "VERIFIED"
}
```


Setup requirements

- Provide a **public HTTPS callback URL** for each webhook type.
- Your endpoint must accept **HTTP POST** requests with a JSON body.
- A **webhook signing secret** is issued from **Dashboard** → **Settings** → **Security** → **Webhooks** for signature verification.

Signature verification

Every callback request includes an `X-Signature` header. You **must verify** this signature before processing the payload.

> Verify over the **raw request body exactly as received** — do **not** `JSON.parse` and then re-serialize. Re-stringifying can change byte-level details (key order, whitespace, unicode escaping) and cause verification to fail even for a genuine callback. Capture the raw body **before** any JSON parsing.

The signature is `HMAC-SHA256(webhook_secret, rawBody)`, hex-encoded:

```
X-Signature = HMAC_SHA256(webhook_secret, <raw request body bytes>)   // lowercase hex
```

Verification (JavaScript / TypeScript)

verify.ts

```ts
import crypto from "crypto";

// `rawBody` is the exact request body string as received (before JSON.parse).
export function verifySignature(signature: string, rawBody: string, secret: string): boolean {
  const computed = crypto
    .createHmac("sha256", secret)
    .update(rawBody, "utf8")
    .digest("hex");

  return crypto.timingSafeEqual(
    Buffer.from(signature.trim().toLowerCase(), "hex"),
    Buffer.from(computed, "hex"),
  );
}
```

Security considerations

- Store callback secrets securely (secret manager, not source control).
- Always use **timing-safe** comparison when verifying.
- Reject the webhook if signature verification fails.
- Do not trust webhook payloads without verification.
- Treat webhooks as **event triggers** and use the status APIs as the source of truth.

Payouts

Initiate bank or UPI payouts and track their lifecycle.

The Payouts API initiates outbound transfers and exposes their state. Both **bank account** and **UPI** rails are supported; the rail is inferred from the fields you provide.

Create payout

POST `{BASE_URL}/account/payouts/initiate`

Request schema: `PayoutCreateRequest`.

Initiates a bank-account or UPI payout for a specified amount and currency.

- If `vpa` is provided, a **UPI payout** is initiated.
- If `account_number` and `bank_ifsc_code` are provided, a **bank-account payout** is initiated.

On success the payout enters `processing` state and can be tracked using the returned `payout_id`.

Request body

Field	Type	Required	Description
`bank_currency`	string	yes	Fiat currency code for the payout (for example, `inr`).
`amount`	string	yes	Requested payout amount. Must be a positive number, sent as a string.
`account_holder_name`	string	yes	Full name of the bank account / UPI holder.
`client_reference_id`	string	yes	Unique client reference (`^[A-Za-z0-9_-]{8,64}$`). Acts as the idempotency key.
`bank_ifsc_code`	string	conditional	Bank IFSC / SWIFT code. Required together with `account_number` for bank payouts.
`account_number`	string	conditional	Bank account number. Required together with `bank_ifsc_code` for bank payouts.
`vpa`	string	conditional	UPI ID of the recipient. Mutually exclusive with `account_number` and `bank_ifsc_code`.
`sender_name`	string	no	Name of the sender. Max 128 chars. Only letters and spaces are kept — any non-`[A-Za-z ]` characters (digits, punctuation) are **stripped server-side**, so send an alphabetic name to avoid surprises.
`sender_phone_number`	string	no	Sender's phone number in E.164 form (`^\+?[1-9]\d{7,14}$` — 8–15 digits, optional leading `+`). Spaces, `(`, `)` and `-` are stripped. e.g. `+919876543210`.
`sender_nationality`	string	no	Sender's nationality (citizenship) as an ISO 3166-1 alpha-3 country code (`^[A-Z]{3}$`), upper-cased. e.g. `IND`.

Field	Type	Required	Description
`sender_address`	object	no	Sender's postal address. Nested, all sub-fields optional: `address`, `city`, `state`, `postal_code`, and `country_code` (ISO 3166-1 alpha-3, `^[A-Z]{3}$`, upper-cased, e.g. `IND`). Independent of `sender_nationality`.
`sender_dob`	string	no	Sender's date of birth as `YYYY-MM-DD`; must be a real calendar date in the past. e.g. `1990-05-17`.
`sender_doc_number`	string	no	Sender's identity-document number (`^[A-Z0-9/-]{4,32}$`), upper-cased with spaces removed. e.g. `ABCDE1234F`.
`sender_doc_type`	string	no	Sender's identity-document type. One of `pan_card`, `aadhaar`, `passport`, `bvn`, `voter_id`, `driving_license`.
`penny_drop_required`	boolean	no	Whether bank-account verification via penny drop should run before payout. Default `true`.
`otc_quote_id`	string	no	When set, the payout is funded by placing an order against this previously-created OTC quote (`POST /market/otc_quotes/new`). The resulting order is embedded in the response under `otc_order`. Omit to settle from existing balance.

> The `sender_*` fields declare the sender's identity for KYC / travel-rule reporting. All are optional and independent — send only the ones you have. What you send is stored under `metadata.sender_info` and echoed back on the payout and status responses; fields you omit are simply absent.

Examples

Request — bank account

```
{
  "bank_currency": "inr",
  "amount": "100",
  "account_holder_name": "Jhon Deo",
  "bank_ifsc_code": "HDFC0000044",
  "account_number": "1234567890",
  "client_reference_id": "BIGBULL0001-2",
  "penny_drop_required": true
}
```

Response — 201 Created


```
{
  "payout_id": "TID2512240955O3GMA37C3EE",
  "client_reference_id": "BIGBULL0001-2",
  "bank_currency": "inr",
  "amount": "100.0",
  "fee": "0.0",
  "payout_status": "processing",
  "status_code": "PROCESSING",
  "status_message": "Payout in Progress",
  "created_at": 1766570103137
}
```


Request — UPI


```
{
  "bank_currency": "inr",
  "amount": "100",
  "account_holder_name": "Jhon Deo",
  "vpa": "user@okaxis",
  "client_reference_id": "BIGBULL0001-2"
}
```


Response — 201 Created


```
{
  "payout_id": "TID2512240955O3GMA37C3EE",
  "client_reference_id": "BIGBULL0001-2",
  "bank_currency": "inr",
  "amount": "100.0",
  "fee": "0.0",
  "payout_status": "processing",
  "status_code": "PROCESSING",
  "status_message": "Payout in Progress",
  "created_at": 1766570103137
}
```

Request — OTC-funded (via an existing quote)


```
{
  "client_reference_id": "TEST00006",
  "bank_currency": "inr",
  "amount": "862.1",
  "account_holder_name": "Sender Name",
  "account_number": "1234567890",
  "bank_ifsc_code": "HDFC0000044",
  "sender_name": "My Name",
  "otc_quote_id": "OQID260622085253PPC823F2"
}
```


Response — 201 Created (with otc_order)


```
{
  "payout_id": "TID260622093449PPA73A980",
  "bank_currency": "inr",
  "beneficiary_id": "WBID250705030227PPB83096",
  "client_reference_id": "TEST00006",
  "amount": "862.1",
  "fee": "0.0",
  "payout_status": "processing",
  "bank_transaction_id": null,
  "remark": "PROCESSING",
  "status_code": "PROCESSING",
  "status_message": "Payout in Progress",
  "sender_name": "MY NAME",
  "created_at": 1782120889585,
  "completed_at": null,
  "otc_order": {
    "order_id": "OOID260622090249PPCCFE9D",
    "market": "usdtinr",
    "client_reference_id": "TID260622093449PPA73A980",
    "amount": "100.05",
    "price": "86.21",
    "fee": "0.0",
    "fee_currency": "inr",
    "tax": "0.0",
    "tax_currency": "inr",
    "side": "sell",
    "order_status": "completed",
    "created_at": 1782118969689
  }
}
```

> When `otc_quote_id` is provided, the payout is funded by placing an order against that previously-created OTC quote. The embedded `otc_order.client_reference_id` is automatically populated with the generated `payout_id`, so you can correlate the two records. Omit `otc_quote_id` to settle from existing balance — in that case `otc_order` is not present in the response.

Responses

Code	Meaning
`201`	Payout created. Check `payout_status` — see the note below.
`400`	Invalid request syntax.
`401`	Unauthorized — see Error codes.
`403`	The account is not permitted to create payouts.
`422`	Validation error, duplicate reference, or OTC-quote error.
`500`	Unexpected server error — the payout may have been created.

> When `otc_quote_id` is set and the OTC order cannot be placed (the price or amount changed, or the deal is no longer available), the payout is cancelled but the response is still `201`, with `payout_status: "rejected"` and `status_code: "UNKNOWN_ERROR"`. The `client_reference_id` is consumed — retry with a new reference and a new quote.

After a `500` or a timeout, call Get payout status with the same `client_reference_id` before retrying.

Error response shape

Error responses return an `errors` array of machine-readable codes.

Field-validation errors (the `invalid_sender_*` codes and any missing / invalid field) are returned **prefixed with the offending parameter name** — e.g. `"sender_phone_number account.payout.invalid_sender_phone_number"`. Match on the code, not the whole string. Business errors such as `duplicate_reference_id` are returned bare.

`422` codes specific to this endpoint:

- `account.payout.duplicate_reference_id_#<client_reference_id>` — a payout with that `client_reference_id` already exists.
- `account.payout.create_in_processing` — a request with the same `client_reference_id` is already being processed (sent again within 5 seconds). Check the payout status instead of retrying.
- `account.payout.insufficient_balance_#<client_reference_id>` — the account balance is too low for the payout. Top up, then retry with a **new** `client_reference_id`.
- `account.market.invalid_client_reference_id` — `client_reference_id` must be 8–64 characters of `A–Z`, `a–z`, `0–9`, `_` or `-`.
- `account.payout.invalid_sender_phone_number` — `sender_phone_number` is not valid E.164.
- `account.payout.invalid_sender_nationality` — `sender_nationality` is not an ISO 3166-1 alpha-3 code.

- `account.payout.sender_nationality_not_allowed` — `sender_nationality` is not an accepted nationality.
- `account.payout.invalid_sender_country_code` — `sender_address.country_code` is not an ISO 3166-1 alpha-3 code.
- `account.payout.invalid_sender_dob` — `sender_dob` is not a real past `YYYY-MM-DD` date.
- `account.payout.invalid_sender_doc_number` — `sender_doc_number` fails the format check.
- `account.payout.invalid_sender_doc_type` — `sender_doc_type` is not a supported document type.

When the request includes `otc_quote_id`, errors from the OTC quote system propagate through (also as `422`):

- `market.otc_quote.not_found` — `otc_quote_id` doesn't match any quote.
- `market.otc_quote.amount_mismatch` — quote amount no longer matches.
- `market.otc_quote.expired` — quote is past its expiry.
- `market.otc_quote.consumed_or_expired` — quote has already been used.

Parameter-combination errors are returned as plain messages, e.g. `vpa, account_number are mutually exclusive` or `account_number, bank_ifsc_code provide all or none of parameters`.

`403` **codes:** `account.fiat_withdraw.not_permitted`, `user.ability.not_permitted`.

```
422 Duplicate reference (bare code)
```

```
{ "errors": ["account.payout.duplicate_reference_id_#BIGBULL0010"] }
```

```
422 Sender validation (field-prefixed)
```

```
{ "errors": ["sender_phone_number account.payout.invalid_sender_phone_number"] }
```

```
422 OTC quote expired (propagated)
```

```
{ "errors": ["market.otc_quote.expired"] }
```

```
401 Unauthorized
```

```
{ "errors": ["authz.invalid_signature"] }
```

Sandbox testing

In the **sandbox** environment the payout outcome is simulated from the **first character of** `account_number`:

`account_number` **starts with**	**Result**
`0` – `2`	✅ Random `PROCESSING` or `SUCCESS`
`3`	⭕ Always `PROCESSING`
`4`	❌ `FAILED` — invalid transaction
`5` – `6`	❌ `FAILED` — provider down
`9`	❌ `FAILED` — transaction terminated

Any other prefix returns a random fallback failure.

Get payout status

GET `{BASE_URL}/account/payouts/status`

Track the progress of a payout using the payout ID or your `client_reference_id`.

Query parameters

Parameter	Type	Notes
`payout_id`	string	The unique identifier of the payout.
`client_reference_id`	string	The client-provided reference ID.
`bank_transaction_id`	string	The transaction ID assigned by the bank.
`bank_currency`	string	Filter by fiat currency.
`page`	integer	Pagination — page number.
`limit`	integer	Pagination — page size.
`ordering`	string	Sort direction: `asc` or `desc` (default `desc`).
`order_by`	string	Sort field: `id`, `created_at`, `updated_at`.
`range`	string	Date filter field: `created` or `updated` (default `created`).
`from`	string	Start datetime (ISO 8601), e.g. `2025-09-30T00:00:00Z`.
`to`	string	End datetime (ISO 8601), e.g. `2025-09-30T23:59:59Z`.

Pagination metadata is returned via response headers: `Total`, `Page`, `Per-Page`.

Responses

Code	Meaning
`200`	Payout status retrieved successfully.

The response is always an array. An empty array (`[]`) means no payout exists for the given filter — for a `client_reference_id`, it is safe to send the payout again with the same reference.

Response

200 OK


```
[
  {
    "payout_id": "TID251224095503GMA37C3EE",
    "bank_currency": "inr",
    "beneficiary_id": "WBID251224092108GMC9757E",
    "client_reference_id": "BIGBULL0001-2",
    "amount": "100.0",
    "fee": "0.0",
    "payout_status": "processing",
    "bank_transaction_id": null,
    "status_code": "PROCESSING",
    "status_message": "Payout in Progress",
    "sender_name": null,
    "sender_phone_number": null,
    "sender_nationality": null,
    "sender_dob": null,
    "sender_doc_number": null,
    "sender_doc_type": null,
    "sender_address": null,
    "created_at": 1766570103137,
    "completed_at": null
  }
]
```


Response fields — `PayoutResponse`

Field	Type	Description
`payout_id`	string	Unique payout identifier issued by the platform.
`bank_currency`	string	Fiat currency of the payout.
`beneficiary_id`	string | null	Linked beneficiary, when applicable.
`client_reference_id`	string	Your idempotency key as supplied on create.
`amount`	string	Payout amount (string-encoded number).
`fee`	string	Fee charged for this payout.
`payout_status`	enum string	Primary state — use this for routing logic. One of `processing`, `succeed`, `rejected`, `refunded`. A `succeed` payout can still move to `refunded` if the bank returns the funds.
`status_code`	enum string	Internal sub-status for operational tracking. Not the primary state — see Payout status codes below for the full enum.
`status_message`	string	Human-readable explanation of the current status.

Field	Type	Description
`remark`	string | null	Free-form note attached to the payout, typically mirroring `status_code` for in-flight payouts.
`bank_transaction_id`	string | null	Transaction reference assigned by the rail bank.
`sender_name`	string | null	Sender name, if supplied on create.
`sender_phone_number`	string | null	Sender phone number (E.164), if supplied on create.
`sender_nationality`	string | null	Sender nationality / citizenship (ISO 3166-1 alpha-3), if supplied on create.
`sender_dob`	string | null	Sender date of birth (`YYYY-MM-DD`), if supplied on create.
`sender_doc_number`	string | null	Sender identity-document number, if supplied on create.
`sender_doc_type`	string | null	Sender identity-document type, if supplied on create.
`sender_address`	object | null	Sender postal address (`address` , `city` , `state` , `postal_code` , `country_code` — alpha-3), if supplied on create.
`created_at`	integer	Creation timestamp, milliseconds since epoch.
`completed_at`	integer | null	Completion timestamp, milliseconds since epoch (same shape as `created_at`). `null` while in flight.
`otc_order`	object | absent	Present only when the payout was funded by an OTC order (request supplied `otc_quote_id`). The embedded order follows the OtcOrderResponse schema; its `client_reference_id` is the generated `payout_id` .

Payout status codes

`status_code` is one of the following values. Use `payout_status` for primary routing logic and `status_code` for finer-grained operational visibility.

`PENDING` · `PROCESSING` · `DEEMED_APPROVED` · `COMPLETED` · `REFUNDED` · `TRANSACTION_FAILED` · `UNKNOWN_ERROR` · `ACCOUNT_BLOCKED` · `ACCOUNT_CLOSED` · `CREDIT_NOT_PERMITTED` · `DAILY_ACCOUNT_LIMIT_EXCEED` · `IMPS_NOT_SUPPORTED` · `INVALID_ACCOUNT_NUMBER` · `INVALID_IFSC_CODE` · `INVALID_VPA` · `INVALID_VPA_FORMAT` · `J&K_REGION_BLOCKED` · `NOT_COMPLIANT` · `NRE_ACCOUNT` · `PROVIDER_ISSUE`

> Treat callbacks as triggers — see Best practices. The status returned from this endpoint is the source of truth for a payout.

Beneficiaries

Register and verify payees before initiating a payout.

The Beneficiaries API registers a payment destination and verifies it before funds can be sent. Both **bank account** (verified via penny drop) and **UPI** beneficiaries are supported.

Create beneficiary (penny-drop verification)

POST `{BASE_URL}/account/beneficiaries/penny_drop`

Request schema: `BeneficiaryCreateRequest`.

Creates and verifies a bank-account or UPI beneficiary.

- If `vpa` is provided, a **UPI beneficiary** is created and verified.
- If `account_number` and `bank_ifsc_code` are provided, a **bank-account beneficiary** is created and verified via penny drop.

The penny-drop transfers a small amount to validate the account details. Once `verification_status` becomes `active`, payouts can be initiated for the beneficiary.

Request body

Field	Type	Required	Description
`bank_currency`	string	yes	Fiat currency code used by the bank (for example, `inr`).
`account_holder_name`	string	yes	The name of the account holder.
`bank_ifsc_code`	string	conditional	The IFSC code of the bank. Required together with `account_number` for bank accounts.
`account_number`	string	conditional	The account number. Required together with `bank_ifsc_code` for bank accounts.
`vpa`	string	conditional	UPI ID of the beneficiary. Mutually exclusive with `account_number` and `bank_ifsc_code`.
`penny_drop_required`	boolean	no	Whether bank-account verification via penny drop should run before the beneficiary is used. Default `true`.
`sync_bank_verification`	boolean	no	When `true`, verification runs immediately. When omitted/false, verification batches.

Examples

Request — bank account

```
{
  "bank_currency": "inr",
  "bank_ifsc_code": "HDFC0000044",
  "account_holder_name": "Jhon Doe",
  "account_number": "1234567890",
  "sync_bank_verification": true
}
```

Response — 201 (pending)

```
{
  "beneficiary_id": "WBID251224092108GMC9757E",
  "bank_currency": "inr",
  "bank_ifsc_code": "HDFC0000044",
  "account_number": "1234567890",
  "account_holder_name": "Jhon Doe",
  "remark": "PENDING",
  "status_code": "PENDING",
  "status_message": "Verification Pending",
  "verification_status": "inactive",
  "verification_data_bank_ref_no": null,
  "verification_data_bank_beneficiary_name": null,
  "verification_data_bank_transaction_remark": "No Record Found",
  "verification_data_bank_transaction_status": "NOT_FOUND"
}
```

Request — UPI

```
{
  "bank_currency": "inr",
  "vpa": "user@okaxis",
  "account_holder_name": "Jhon Doe"
}
```

Response — 201 (verified synchronously)


```
{
  "beneficiary_id": "WVID260302082955LD124BF6",
  "bank_currency": "inr",
  "vpa": "user@okaxis",
  "account_holder_name": "Jhon Doe",
  "remark": "VERIFIED",
  "status_code": "VERIFIED",
  "status_message": "Verification Completed",
  "verification_status": "active",
  "verification_data_bank_ref_no": null,
  "verification_data_bank_beneficiary_name": "John Doe",
  "verification_data_bank_transaction_remark": null,
  "verification_data_bank_transaction_status": "VERIFIED"
}
```


Responses

Code	Meaning
`201`	Beneficiary created; penny drop initiated. If the beneficiary already exists, the existing record is returned.
`401`	Unauthorized — see Error codes.
`422`	Validation error, or the beneficiary could not be created.

Error response shape

Errors return an `errors` array of machine-readable codes.

`422` codes:

- `account.bank_account.create_in_processing` — the same bank account is already being created. Retry after a few seconds.
- `account.vpa_account.create_in_processing` — the same UPI ID is already being created. Retry after a few seconds.
- `account.bank_account.failed_to_create` — the beneficiary could not be created.

422 Already being created


```
{ "errors": ["account.bank_account.create_in_processing"] }
```


401 Unauthorized


```
{ "errors": ["authz.invalid_signature"] }
```

Sandbox testing

In the **sandbox** environment the penny-drop outcome is simulated from the **first character of** `account_number`. Creation always returns **HTTP 201** with `verification_status: processing`; the **terminal** result is delivered **asynchronously** via the beneficiary webhook callback (and reflected on the status endpoint):

`account_number` **starts with**	`verification_status`	`status_code`
`0` – `9`	`active`	`VERIFIED`
`A`	`rejected`	`INVALID_ACCOUNT_NUMBER`
`B`	`rejected`	`INVALID_IFSC_CODE`
`C`	`rejected`	`ACCOUNT_BLOCKED`
`D`	`rejected`	`NRE_ACCOUNT`
`E`	stays `processing`	`PROCESSING` — daily limit; only rejected later during DB cleanup

There is no synchronous failure response — every prefix returns **HTTP 201**, and the pass/fail verdict arrives on the callback. An `E` beneficiary never resolves on its own; it stays `processing` until a later DB-cleanup pass may reject it.

Get beneficiary status

GET `{BASE_URL}/account/beneficiaries/status`

Query beneficiary information by beneficiary ID, account number, or UPI ID. Once `verification_status` becomes `active`, payouts can be initiated.

Query parameters

Parameter	Type	Notes
`beneficiary_id`	string	The unique identifier of the beneficiary.
`vpa`	string	UPI ID. Mutually exclusive with `account_number` and `bank_ifsc_code`.
`account_number`	string	Bank account number. Mutually exclusive with `vpa`.
`bank_ifsc_code`	string	IFSC code. With `account_number`: exact match. Without: returns all records for the IFSC.
`bank_currency`	string	Filter by fiat currency.
`page`	integer	Pagination — page number.
`limit`	integer	Pagination — page size.
`ordering`	string	Sort direction: `asc` or `desc` (default `desc`).
`order_by`	string	Sort field: `id`, `created_at`, `updated_at`.
`range`	string	Date filter field: `created` or `updated` (default `created`).

Parameter	Type	Notes
`from` / `to`	string	ISO 8601 datetime bounds.

Responses

Code	Meaning
`200`	Beneficiary status retrieved successfully.

Response

`200 OK — bank account`


```
[
  {
    "beneficiary_id": "WBID251224092108GMC9757E",
    "bank_currency": "inr",
    "bank_ifsc_code": "HDFC0000044",
    "account_number": "1234567890",
    "account_holder_name": "Jhon Doe",
    "remark": "PENDING",
    "status_code": "PENDING",
    "status_message": "Verification Pending",
    "verification_status": "inactive",
    "verification_data_bank_ref_no": null,
    "verification_data_bank_beneficiary_name": null,
    "verification_data_bank_transaction_remark": "No Record Found",
    "verification_data_bank_transaction_status": "NOT_FOUND"
  }
]
```


Response fields — `BeneficiaryResponse`

Field	Type	Description
`beneficiary_id`	string	Unique identifier for the beneficiary.
`bank_currency`	string	Fiat currency.
`bank_ifsc_code`	string	IFSC code (present for bank-account beneficiaries).
`account_number`	string	Account number (present for bank-account beneficiaries).
`vpa`	string	UPI ID (present for UPI beneficiaries).
`account_holder_name`	string	Holder name as supplied on create.
`remark`	string | null	Additional notes from the verification step.

Field	Type	Description
`status_code`	enum string	Internal sub-status. See Beneficiary status codes for the full enum.
`status_message`	string	Human-readable verification state.
`verification_status`	enum string	Primary state. One of `pending`, `processing`, `inactive`, `active`, `rejected`, `archived`. Payouts can be initiated only when `active`.
`verification_data_bank_ref_no`	string | null	Bank reference number for the penny-drop transaction.
`verification_data_bank_beneficiary_name`	string | null	Beneficiary name returned by the bank.
`verification_data_bank_transaction_remark`	enum string | null	Bank remark after verification. One of `Transaction Successful`, `No Record Found`.
`verification_data_bank_transaction_status`	enum string | null	Bank-side transaction status. One of `PROCESSING`, `VERIFIED`, `NOT_FOUND`, `FAILED`.

Beneficiary status codes

`status_code` is one of the following values. Use `verification_status` for primary routing logic and `status_code` for finer-grained operational visibility.

`PENDING` · `PROCESSING` · `VERIFIED` · `ACCOUNT_BLOCKED` · `ACCOUNT_CLOSED` · `CREDIT_NOT_PERMITTED` · `DAILY_ACCOUNT_LIMIT_EXCEED` · `IMPS_NOT_SUPPORTED` · `INVALID_ACCOUNT_NUMBER` · `INVALID_IFSC_CODE` · `INVALID_VPA` · `INVALID_VPA_FORMAT` · `J&K_REGION_BLOCKED` · `NOT_COMPLIANT` · `NRE_ACCOUNT` · `PROVIDER_ISSUE`

Balances

Fetch fiat and crypto balances across the account.

Get account balances

GET `{BASE_URL}/account/balances`

Retrieves account balance information. By default this endpoint returns balances for **all currencies**.

Single-currency lookup

Append a currency code to the path to fetch a single balance:

```
GET /account/balances/inr
```

Path parameters

Parameter	Type	Notes
`currency`	string	Optional. Currency code (for example, `inr`, `usdt`).

Query parameters

Parameter	Type	Notes
`wallet_type`	string	Filter balances by wallet type (for example, `spot`).
`currency_id`	string	Optional. Filter the list by currency code (for example, `inr`).

Responses

Code	Meaning
`200`	Balance information retrieved successfully.
`422`	Currency not found (single-currency lookup).

GET /account/balances


```
[
  {
    "currency": "inr",
    "balance": "15000.50",
    "locked": "2000.00",
    "escrow": "500.00",
    "wallet_type": "spot"
  },
  {
    "currency": "usdt",
    "balance": "250.00",
    "locked": "0.00",
    "escrow": "0.00",
    "wallet_type": "spot"
  }
]
```


GET /account/balances/inr


```
{
  "currency": "inr",
  "balance": "15000.50",
  "locked": "2000.00",
  "escrow": "500.00",
  "wallet_type": "spot"
}
```


Response fields — `BalanceResponse`

Field	Type	Description
`currency`	string	Currency code (for example, `inr`, `usdt`).
`balance`	string	Total balance held for this currency (string-encoded number).
`locked`	string	Portion of the balance committed to in-flight payouts.
`escrow`	string	Portion held in escrow against pending settlement.
`wallet_type`	string	Wallet type — currently `spot` for all currencies.

OTC quotes & orders

Request FX prices and execute conversion orders.

The OTC API exposes a two-step flow: **request a quote** with a guaranteed rate, then **place an order** against that quote. You can also create an order directly without a quote when slippage protection isn't needed.

Create OTC quote

POST `{BASE_URL}/market/otc_quotes/new`

Request schema: `OtcQuoteCreate`.

Creates a new OTC quote for the given market and side. Quotes are time-bound and must be used before they expire.

Request body

Field	Type	Required	Notes
`req_amount`	string	yes	Requested quote amount, denominated in `req_currency`.
`req_currency`	string	no	Currency the `req_amount` is in. Optional — defaults to the market's base currency (e.g. `usdt` for `usdtinr`) when omitted. Typically set to the fiat side (e.g. `inr`).
`market`	string	yes	Trading pair (for example, `usdtinr`).
`side`	string	yes	Quote side — currently `sell` only.
`client_reference_id`	string	no	Optional your-side reference. Echoed back on the quote and on orders placed against it.

Request

```
{
  "req_amount": "8621",
  "req_currency": "inr",
  "market": "usdtinr",
  "side": "sell",
  "client_reference_id": "QUOTE-TEST-001"
}
```

Response — 201 Created


```
{
  "quote_id": "OQID260101143053GM990BB3",
  "market": "usdtinr",
  "client_reference_id": "QUOTE-TEST-001",
  "req_amount": "8621.0",
  "req_currency": "inr",
  "allotted_amount": "100.05",
  "avg_price": "86.17",
  "side": "sell",
  "quote_status": "active",
  "remark": null,
  "otc_orders": [],
  "fx_rate": null,
  "fx_provider": null,
  "created_at": 1766570103137,
  "expired_at": 1766570703000
}
```


Responses

Code	Meaning
`201`	OTC quote created successfully.
`422`	Invalid request or duplicate reference.

The 201 response body uses the `OtcQuoteResponse` schema — see field table under Get OTC quote status. A freshly created quote is `active` with an empty `otc_orders` array until an order is placed against it.

422 error codes

- `market.otc_quote.duplicate_reference_id_#<client_reference_id>` — a quote with that `client_reference_id` already exists.
- `market.otc_quote.invalid_client_reference_id` — `client_reference_id` must be 8–64 characters of `A–Z`, `a–z`, `0–9`, `_` or `-`.
- `market.otc_quote.failed_to_create` — the quote could not be created.

422 Duplicate reference


```
{ "errors": ["market.otc_quote.duplicate_reference_id_#TEST-00X-3"] }
```


Get OTC quote status

GET `{BASE_URL}/market/otc_quotes/status`

Retrieves the list of OTC quotes with their current state.

Query parameters

Parameter	Type	Notes
`quote_id`	string	The unique identifier of the quote.
`market`	string	Filter quotes by market (for example, `usdtinr`).
`client_reference_id`	string	Filter by the client reference supplied when the quote was created.
`page`	integer	Pagination — page number.
`limit`	integer	Pagination — page size.
`ordering`	string	Sort direction: `asc` or `desc` (default `desc`).
`order_by`	string	Sort field: `id`, `created_at`, `updated_at`.
`range`	string	Date filter field: `created` or `updated` (default `created`).
`from` / `to`	string	ISO 8601 datetime bounds.

Response fields — `OtcQuoteResponse`

Field	Type	Description
`quote_id`	string	Unique quote identifier.
`client_reference_id`	string | null	Client reference supplied on create, if any.
`market`	string	Trading pair (for example, `usdtinr`).
`req_amount`	string	Amount originally requested, denominated in `req_currency`.
`req_currency`	string	Currency the `req_amount` is denominated in.
`allotted_amount`	string	Amount the platform allotted for the quote (in the opposite side of the market).
`avg_price`	string	Average execution price.
`side`	enum string	Quote side. Currently only `sell`.
`quote_status`	enum string	Current state. One of `active`, `expired`, `consumed`.
`remark`	string | null	Optional remark or system note.
`otc_orders`	array	Orders that have been created from this quote (each follows `OtcOrderResponse`).
`fx_rate`	string | null	Foreign-exchange rate recorded on the deal this quote was priced from; `null` when none.
`fx_provider`	string | null	Source the `fx_rate` was quoted from; `null` when none.
`created_at`	integer	Creation timestamp, milliseconds since epoch.
`expired_at`	integer	Expiry timestamp, milliseconds since epoch.

Place OTC order using a quote

PATCH `{BASE_URL}/market/otc_quotes/place_order/{quote_id}`

Path template: `/market/otc_quotes/place_order/{quote_id}`. Places an OTC order against an active quote. Fails if the quote is expired or the amount has changed since the quote was issued.

Path parameters

Parameter	Type	Required	Notes
`quote_id`	string	yes	The quote ID, for example `OQID260101143053GM990BB3`.

Request body (optional)

Field	Type	Required	Notes
`client_reference_id`	string	no	Optional reference for the order being created. If omitted, the order inherits the quote's `client_reference_id` (when set), otherwise the platform generates one.

A successful call returns the **updated quote** (`OtcQuoteResponse`) — now `consumed` — with the created order embedded under `otc_orders`, rather than a bare order object.

Request (optional body)

```
{
  "client_reference_id": "ORDER-TEST-001"
}
```

Response — 200 OK


```
{
  "quote_id": "OQID260101143053GM990BB3",
  "market": "usdtinr",
  "client_reference_id": "QUOTE-TEST-001",
  "req_amount": "8621.0",
  "req_currency": "inr",
  "allotted_amount": "100.05",
  "avg_price": "86.17",
  "side": "sell",
  "quote_status": "consumed",
  "remark": null,
  "otc_orders": [
    {
      "order_id": "OOID260101143512GMB47A21",
      "market": "usdtinr",
      "client_reference_id": "ORDER-TEST-001",
      "amount": "100.05",
      "price": "86.20",
      "order_status": "completed",
      "created_at": 1766570103137
    }
  ],
  "fx_rate": null,
  "fx_provider": null,
  "created_at": 1766570103137,
  "expired_at": 1766570703000
}
```


Responses

Code	Meaning
`200`	OTC order placed successfully.
`422`	Quote error (expired or stale).

422 error codes

`422` returns an `errors` array of machine-readable codes:

- `market.otc_quote.not_found` — the `quote_id` doesn't match any quote.
- `market.otc_quote.expired` — quote is past its expiry timestamp.
- `market.otc_quote.consumed_or_expired` — an order has already been placed against the quote.
- `market.otc_quote.cannot_place_order` — the order could not be placed against the quote.

```
422 Cannot place order
```

```
{ "errors": ["market.otc_quote.cannot_place_order"] }
```

```
422 Quote expired
```

```
{ "errors": ["market.otc_quote.expired"] }
```

Get OTC orders

GET `{BASE_URL}/market/otc_orders`

Retrieves OTC orders with optional filtering.

Query parameters

Parameter	Type	Notes
`order_id`	string	Filter by specific order ID.
`client_reference_id`	string	Filter by client-provided reference ID.
`market`	string	Filter by trading pair.
`page`, `limit`	integer	Pagination.
`ordering`, `order_by`	string	Sorting.
`range`, `from`, `to`	string	Date filtering.

Response fields — `OtcOrderResponse`

Field	Type	Description
`order_id`	string	Unique OTC order identifier.
`market`	string	Trading pair.
`client_reference_id`	string | null	Reference ID supplied on create, if any.
`amount`	string	Executed amount in the market's **base** currency (USDT for `usdtinr`), not in `req_currency`.
`price`	string	Executed price.
`fee`	string	Fee charged for the order.
`fee_currency`	string	Currency in which the fee is charged.
`tax`	string	Tax charged on the order.
`tax_currency`	string	Currency in which the tax is charged.
`side`	enum string	Order side. Currently only `sell`.
`order_status`	string	Current order status.

Field	Type	Description
`fx_rate`	string | null	Foreign-exchange rate recorded on the deal this order traded against; `null` when none.
`fx_provider`	string | null	Source the `fx_rate` was quoted from; `null` when none.
`created_at`	integer	Creation timestamp, milliseconds since epoch.

Create OTC order (without a quote)

POST `{BASE_URL}/market/otc_orders`

Request schema: `OtcOrderCreate`.

Creates an OTC order directly, without first requesting a quote. The trade executes at the current market rate; use the quote flow when you need a guaranteed rate locked in.

Request body

Field	Type	Required	Notes
`market`	string	yes	Trading pair (for example, `usdtinr`).
`amount`	string	yes	Amount in the currency named by `req_currency`.
`req_currency`	string	no	Currency the `amount` is in. Optional — defaults to the market's base currency (e.g. `usdt` for `usdtinr`) when omitted. Typically set to the fiat side (e.g. `inr`).
`side`	string	yes	Order side — currently `sell` only.
`client_reference_id`	string	yes	Client-provided reference ID (`^[a-zA-Z0-9_-]{8,64}$`). Acts as the idempotency key — repeating an id returns the existing order(s).

Request

```
{
  "amount": "8625",
  "req_currency": "inr",
  "market": "usdtinr",
  "side": "sell",
  "client_reference_id": "TEST00004"
}
```

Response — 201 Created (array)


```
[
  {
    "order_id": "OOID260101143512GMB47A21",
    "market": "usdtinr",
    "client_reference_id": "TEST00004",
    "amount": "100.05",
    "price": "86.20",
    "fee": "0.0",
    "fee_currency": "inr",
    "tax": "0.0",
    "tax_currency": "inr",
    "side": "sell",
    "order_status": "completed",
    "fx_rate": null,
    "fx_provider": null,
    "created_at": 1766570103137
  }
]
```


Responses

Code	Meaning
`201`	OTC order created successfully.

Unlike the quote flow, this endpoint returns an **array** of `OtcOrderResponse` objects — today always one element, so read `response[0]`. Repeating a `client_reference_id` returns the same array (the existing order); no second trade is made.

The response `amount` is in the market's **base** currency, not in `req_currency`: a request for `"amount": "8625"`, `"req_currency": "inr"` at a price of `86.20` returns `"amount": "100.05"` (USDT). Multiply `amount × price` to get the fiat value.

Utilities

Public health and time endpoints for liveness checks and clock alignment.

Unauthenticated utility endpoints under `/public`. No API key is required.

Health check

GET `{BASE_URL}/public/health`

Liveness probe — returns the string `ok` when the service is up.

Responses

Code	Meaning
`200`	Service is healthy.

Response — 200 OK

```
ok
```

Server timestamp

GET `{BASE_URL}/public/timestamp`

Returns the server's current time as a Unix timestamp (seconds). Useful for aligning request nonces and signatures with server time.

Responses

Code	Meaning
`200`	Current server time.

Response — 200 OK

```
1735689600
```